BÁO CÁO THỰC HÀNH KIẾN TRÚC MÁY TÍNH

LAB 2: BRANCHES AND PROCEDURES (LỆNH RẼ NHÁNH VÀ THỦ TỤC)

Thông tin sinh viên	Chi tiết
Họ và tên:	Đoàn Duy Tùng
MSSV:	2033364
Lớp:	L01
Môn học:	Thực hành Kiến trúc Máy tính (CO2008)
Khoa / Trường:	Khoa Khoa học & Kỹ thuật Máy tính - ĐH Bách Khoa ĐHQG TP.HCM
Ngày thực hiện:	23/10/2026

Danh mục file mã nguồn nộp bài:

- `ex1.asm`: Khai báo mảng 5 phần tử, kiểm tra chia hết cho 4 và làm tròn gần nhất.
- `ex2.asm`: Tìm Ước chung lớn nhất (GCD) và Bội chung nhỏ nhất (LCM).
- `ex3.asm`: Tìm giá trị lớn thứ nhì trong mảng 10 phần tử và in toàn bộ các chỉ số.
- `ex4.asm`: Tính giai thừa $N!$ bằng thủ tục đệ quy và quản lý Stack Frame.

Bài 1 (Exercise 1)

Yêu cầu đề bài: 1. Khai báo mảng số nguyên gồm 5 phần tử nhập từ bàn phím. 2. Kiểm tra từng phần tử xem có chia hết cho 4 hay không. 3. Nếu không chia hết cho 4, chuyển phần tử đó thành số chia hết cho 4 gần nó nhất (ưu tiên làm tròn xuống nếu khoảng cách bằng nhau). 4. In mảng kết quả sau khi xử lý ra màn hình terminal.

1. Phương pháp giải quyết:

- **Xét số dư khi chia cho 4:** Với số nguyên x, khi chia cho 4 ta có số dư $r = x \bmod 4 \in \{0, 1, 2, 3\}$:
  - $r = 0$: Số đã chia hết cho 4, giữ nguyên giá trị.
  - $r = 1$: Khoảng cách đến bội số dưới là 1, đến bội số trên là 3 $\implies$ làm tròn xuống ($x = x - 1$).
  - $r = 2$: Khoảng cách hai bên bằng nhau (đều là 2), ưu tiên làm tròn xuống theo đề bài $\implies$ làm tròn xuống ($x = x - 2$).
  - $r = 3$: Khoảng cách đến bội số trên là 1, đến bội số dưới là 3 $\implies$ làm tròn lên ($x = x + 1$).
- **Xử lý số nguyên âm:** Trong tập lệnh MIPS, lệnh `rem` với số bị chia âm sẽ trả về phần dư âm (ví dụ: $-34 \div 4$ dư -2). Để quy tắc làm tròn xuống trên trục số thực áp dụng chính xác cho cả số âm (-34 làm tròn xuống -36), chương trình chuẩn hóa: nếu $r < 0$ thì cộng thêm 4 để đưa số dư về đoạn $[0, 3]$.

- **Xử lý tràn số cực đại:** Khi phần tử có giá trị $2^{31} - 1$ (chia 4 dư 3), nếu làm tròn lên ($+1$) sẽ vượt quá giới hạn số nguyên 32-bit có dấu gây lỗi phần cứng `Arithmetic overflow`. Chương trình kiểm tra điều kiện trước khi cộng để báo lỗi an toàn.
- **Quy ước thanh ghi:**
  - `$s0`: Địa chỉ cơ sở mảng `array` trong RAM.
  - `$s1`: Số phần tử ($N = 5$).
  - `$s2`: Biến đếm vòng lặp i ($0 \leq i < 5$).
  - `$t1`: Giá trị phần tử `arr[i]`.
  - `$t2`: Số dư $r = \text{arr}[i] \bmod 4$.

2. Đoạn mã Assembly MIPS then chốt:

```
    # Đọc phần tử arr[i] từ RAM và tính số dư chia cho 4
    lw   $t1, 0($t0)            # $t1 = arr[i]
    rem  $t2, $t1, 4            # $t2 = arr[i] % 4

    # Chuẩn hóa số dư về [0, 3] cho cả số âm
    bgez $t2, check_rules
    addi $t2, $t2, 4

check_rules:
    beqz $t2, next_element      # r == 0: đã chia hết cho 4, giữ nguyên
    li   $t3, 1
    beq  $t2, $t3, case_sub_1   # r == 1: trừ 1
    li   $t3, 2
    beq  $t2, $t3, case_sub_2   # r == 2: ưu tiên làm tròn xuống -> trừ 2

    # r == 3: kiểm tra chống tràn số 32-bit rồi cộng 1
    li   $t3, 2147483644
    bgt  $t1, $t3, error_overflow
    addi $t1, $t1, 1
    j    save_and_next

case_sub_1:
    addi $t1, $t1, -1
    j    save_and_next

case_sub_2:
    addi $t1, $t1, -2

save_and_next:
    sw   $t1, 0($t0)            # Ghi đè giá trị đã làm tròn vào mảng
```

3. Kết quả kiểm thử trên MARS 4.5:

STT	Mục đích kiểm thử	Dữ liệu đầu vào	Kết quả thực tế trên MARS	Đánh giá
TC01	Ví dụ đề bài	`35, 34, 33, 32, 36`	`Modified array: 36 32 32 32 36`	**PASS**
TC02	Toàn bộ là bội số của 4	`0, 4, 8, 12, 16`	`Modified array: 0 4 8 12 16`	**PASS**
TC03	Dải số âm & ưu tiên tròn xuống	`-35, -34, -33, -32, -4`	`Modified array: -36 -36 -32 -32 -4`	**PASS**
TC04	Hỗn hợp âm dương & số 0	`1, 2, 3, -1, -2`	`Modified array: 0 0 4 0 -4`	**PASS**
TC05	Ngoại lệ tràn số INT_MAX	`2147483647, 4, 8, 12, 16`	`Error: Arithmetic overflow! Element exceeds 32-bit signed integer limit when rounding up.`	**PASS**

Bài 2 (Exercise 2)

Yêu cầu đề bài: Viết chương trình MIPS yêu cầu người dùng nhập hai số nguyên dương a và b (kiểm tra điều kiện $a > 0, b > 0$). In ra ước chung lớn nhất (GCD) và bội chung nhỏ nhất (LCM). Câu nhắc: `"a = "` và `"b = "`. Ví dụ: $a = 6, b = 45 \Longrightarrow \text{GCD} = 3, \text{LCM} = 90$.

1. Phương pháp giải quyết:

- **Thuật toán Euclid tính GCD:** Sử dụng thuật toán Euclid dạng vòng lặp lặp (`while` loop) trong thủ tục lá `gcd` :

$$
\begin{aligned}
&\text{while } (b \neq 0)\,\{\\
&\quad r = a \bmod b;\\
&\quad a = b;\\
&\quad b = r;\\
&\}\\
&\text{return } a;
\end{aligned}
$$

- **Tối ưu kiến trúc (Leaf Procedure):** Do thủ tục `gcd` là thủ tục lá (không gọi bất kỳ hàm con nào bên trong), chương trình không cần sử dụng ngăn xếp Stack (`$sp`), giúp tiết kiệm chu kỳ bộ nhớ và loại trừ hoàn toàn nguy cơ tràn ngăn xếp.
- **Tính LCM và kiểm soát tràn số 32-bit:** Bội chung nhỏ nhất được tính theo công thức $\text{LCM} = \frac{a \times b}{\text{GCD}}$. Để hạn chế nguy cơ tràn số khi tích $a \times b$ quá lớn, chương trình thực hiện phép chia trước: lấy (a/GCD) rồi mới nhân với b. Lệnh `mult` thực hiện phép nhân 32-bit:
  - Nếu thanh ghi kết quả phần cao $HI \neq 0$, hoặc bit dấu của phần thấp $LO < 0$ (bị lật dấu sang âm), chương trình lập tức nhận diện tràn số và báo lỗi.

- **Kiểm tra đầu vào:** Sử dụng lệnh rẽ nhánh `blez $s0` và `blez $s1` để chặn các trường hợp $a \leq 0$ hoặc $b \leq 0$, in thông báo lỗi và kết thúc chương trình an toàn.
- **Quy ước thanh ghi:**
  - `$s0, $s1`: Lưu hai số nguyên dương a và b.
  - `$s2`: Lưu kết quả $\text{GCD}(a, b)$.
  - `$s3`: Lưu kết quả $\text{LCM}(a, b)$.
  - `$a0, $a1`: Đối số truyền vào thủ tục `gcd`.
  - `$v0`: Kết quả trả về từ thủ tục `gcd`.

2. Đoạn mã Assembly MIPS then chốt:

```
# 1. Bắt ngoại lệ số không dương: a <= 0 hoặc b <= 0
    blez $s0, error_input       # Nếu a <= 0 -> Báo lỗi
    blez $s1, error_input       # Nếu b <= 0 -> Báo lỗi

# 2. Tính LCM và bẫy tràn số 32-bit
    div  $s0, $s2               # $t0 = a / GCD
    mflo $t0
    mult $t0, $s1               # mult lưu 32-bit cao vào HI, 32-bit thấp vào LO
    mfhi $t5
    mflo $s3
    bnez $t5, error_lcm_ovf     # HI != 0 -> Báo lỗi tràn số 32-bit!
    bltz $s3, error_lcm_ovf     # Bit dấu bị lật âm -> Báo lỗi!

# 3. Thủ tục tính GCD bằng vòng lặp (Leaf Procedure)
gcd:
gcd_loop:
    beqz $a1, gcd_done          # Nếu b == 0 thì dừng vòng lặp, kết quả là a
    rem  $t0, $a0, $a1          # $t0 = a % b
    move $a0, $a1               # a mới = b
    move $a1, $t0               # b mới = a % b
    j    gcd_loop               # Lặp lại bước chia lấy dư

gcd_done:
    move $v0, $a0               # Trả về kết quả GCD = a trong $v0
    jr   $ra                    # Quay về hàm gọi main
```

3. Kết quả kiểm thử trên MARS 4.5:

STT	Mục đích kiểm thử	Dữ liệu vào (a, b)	Kết quả thực tế trên MARS	Đánh giá
TC01	Ví dụ đề bài	$a = 6, b = 45$	`GCD=3, LCM=90`	**PASS**
TC02	a chia hết cho b	$a = 20, b = 5$	`GCD=5, LCM=20`	**PASS**
TC03	Hai số nguyên tố cùng nhau	$a = 7, b = 13$	`GCD=1, LCM=91`	**PASS**
TC04	Hai số bằng nhau	$a = 15, b = 15$	`GCD=15, LCM=15`	**PASS**
TC05	Trường hợp biên có số 1	$a = 1, b = 100$	`GCD=1, LCM=100`	**PASS**
TC06	Ngoại lệ $a = 0$	$a = 0, b = 10$	`Error: Both a and b must be positive integers (> 0)!`	**PASS**
TC07	Ngoại lệ $b < 0$	$a = 10, b = -5$	`Error: Both a and b must be positive integers (> 0)!`	**PASS**
TC08	Ngoại lệ tràn số 32-bit LCM	$a = 100000, b = 99999$	`Error: Arithmetic overflow! LCM value exceeds 32-bit signed integer limit.`	**PASS**

Bài 3 (Exercise 3)

Yêu cầu đề bài: Viết chương trình MIPS tìm phần tử lớn thứ nhì trong mảng 10 số nguyên nhập từ bàn phím và in tất cả các vị trí chỉ số (index) của phần tử này. Câu nhắc: `"Please insert a element: "`. Ví dụ: `1, 2, 7, -7, 3, 7, 4, 6, 8, 7` $\Longrightarrow$ `Second largest value is 7, found in index 2, 5, 9.`

1. Phương pháp giải quyết:

Chương trình sử dụng giải thuật quét 3 lượt với độ phức tạp thời gian $O(N)$ để không làm thay đổi thứ tự và cấu trúc ban đầu của mảng: - **Lượt 1 (Tìm số lớn nhất `max1`):** Khởi tạo `max1 = arr[0]`, duyệt qua các phần tử còn lại từ chỉ số 1 đến 9 để cập nhật giá trị lớn nhất tuyệt đối. - **Lượt 2 (Tìm số lớn thứ nhì `max2`):** Duyệt lại mảng, chỉ xét các phần tử thỏa mãn điều kiện nghiêm ngặt nhỏ hơn `max1` (`arr[i] < max1`). Chương trình sử dụng cờ hiệu `has_max2 = 0`: - Khi gặp phần tử đầu tiên nhỏ hơn `max1`, gán giá trị đó làm mốc ban đầu cho `max2` và đặt `has_max2 = 1`. - Cơ chế cờ hiệu giúp giải thuật hoạt động chính xác với cả mảng chứa toàn số âm (tránh lỗi khởi tạo mốc giả bằng 0). - Nếu duyệt hết mảng mà `has_max2` vẫn bằng 0 (tất cả 10 phần tử bằng nhau), chương trình báo lỗi không tồn tại `max2`. - **Lượt 3 (In kết quả và các chỉ số):** Duyệt mảng lần cuối, so sánh từng phần tử với `max2`. Nếu phần tử bằng `max2` thì in chỉ số i của nó, sử dụng biến đếm để

chèn dấu phẩy `", "` ngăn cách chuẩn xác. - **Quy ước thanh ghi:** - `$s0`: Địa chỉ cơ sở mảng `array` trong RAM (được căn lề bằng `.align 2`). - `$s1`: Số phần tử ($N = 10$). - `$s2`: Giá trị lớn nhất (`max1`). - `$s3`: Giá trị lớn thứ nhì (`max2`). - `$s4`: Cờ hiệu `has_max2` (0 hoặc 1). - `$s5`: Đếm số lượng chỉ số đã in để định dạng dấu phẩy.

2. Đoạn mã Assembly MIPS then chốt:

```
# Lượt 2: Tìm max2 nghiêm ngặt nhỏ hơn max1
find_max2_start:
    li   $s4, 0                # has_max2 = 0
    li   $t0, 0                # i = 0
max2_loop:
    bge  $t0, $s1, check_valid_max2
    sll  $t1, $t0, 2
    add  $t1, $s0, $t1
    lw   $t2, 0($t1)           # $t2 = arr[i]
    bge  $t2, $s2, skip_max2   # Bỏ qua nếu arr[i] >= max1
    beqz $s4, init_first_max2  # Gán mốc đầu tiên nếu has_max2 == 0
    ble  $t2, $s3, skip_max2   # Không cập nhật nếu arr[i] <= max2
    move $s3, $t2              # Cập nhật max2 = arr[i]
    j    skip_max2

init_first_max2:
    move $s3, $t2              # max2 = arr[i]
    li   $s4, 1                # has_max2 = 1

skip_max2:
    addi $t0, $t0, 1
    j    max2_loop

check_valid_max2:
    beqz $s4, error_all_equal  # Nếu has_max2 == 0 -> Báo lỗi cả mảng bằng nhau
```

3. Kết quả kiểm thử trên MARS 4.5:

STT	Mục đích kiểm thử	Dữ liệu vào (10 phần tử)	Kết quả thực tế trên MARS	Đánh giá
TC01	Đề bài (`max2` lặp 3 lần)	`1, 2, 7, -7, 3, 7, 4, 6, 8, 7`	`Second largest value is 7, found in index 2, 5, 9.`	**PASS**
TC02	`max2` xuất hiện đúng 1 lần	`10, 20, 30, 40, 50, 60, 70, 80, 90, 100`	`Second largest value is 90, found in index 8.`	**PASS**
TC03	`max1` lặp lại nhiều lần	`99, 99, 99, 50, 20, 10, 50, 5, 2, 1`	`Second largest value is 50, found in index 3, 6.`	**PASS**
TC04	Mảng chứa toàn số âm	`-10, -5, -20, -5, -30, -5, -40, -50, -60, -70`	`Second largest value is -10, found in index 0.`	**PASS**
TC05	Ngoại lệ 10 số bằng nhau	`5, 5, 5, 5, 5, 5, 5, 5, 5, 5`	`Error: All elements are equal, second largest value does not exist!`	**PASS**

Bài 4 (Exercise 4)

Yêu cầu đề bài: Viết chương trình MIPS nhận một số nguyên không âm N từ người dùng (kiểm tra tính hợp lệ $N \geq 0$). In giai thừa $N!$ ra màn hình. Nếu nhập số âm (ví dụ: -2), in ra: `A factorial for your number can not be found.`

1. Phương pháp giải quyết:

- **Giải thuật đệ quy tính Giai thừa:** Thủ tục đệ quy `factorial` được cài đặt theo định nghĩa toán học:

$$N! = \begin{cases} 1 & \text{khi } N \leq 1 \text{ (điều kiện dừng cho } 0! = 1, 1! = 1) \\ N \times (N-1)! & \text{khi } N > 1 \text{ (bước đệ quy)} \end{cases}$$

- **Quản lý Khung ngăn xếp (Stack Frame):** Do mỗi lần gọi lồng nhau qua `jal factorial` thanh ghi `$ra` sẽ bị ghi đè, thủ tục cấp phát 8 bytes trên Stack trước mỗi bước đệ quy:
  - `4($sp)` : Lưu địa chỉ trở về `$ra` .
  - `0($sp)` : Lưu giá trị tham số n hiện tại (`$a0`) để dùng cho phép nhân sau khi hàm con trả về.
  - Sau khi hoàn thành lời gọi con, thủ tục rút `$a0` và `$ra` ra khỏi Stack, hoàn trả 8 bytes (`addi $sp, $sp, 8`) rồi thực hiện `mul $v0, $a0, $v0` và nhảy về bằng `jr $ra` .
- **Kiểm tra đầu vào và bẫy tràn số:**

- Nếu $N < 0$: Lệnh `bltz $s0` chuyển đến khối in thông báo lỗi `A factorial for your number can not be found.`
- Giới hạn biểu diễn 32-bit có dấu là $[-2^{31}, 2^{31} - 1]$. Vì $12! = 479,001,600$ nằm trong phạm vi 32-bit nhưng $13! = 6,227,020,800$ sẽ gây tràn số làm lật dấu sang âm, chương trình kiểm tra nếu $N > 12$ thì báo lỗi tràn số ngay từ đầu.

- **Quy ước thanh ghi:**
  - `$s0`: Lưu số nguyên N nhập vào.
  - `$s1`: Lưu kết quả giai thừa $N!$.
  - `$a0`: Tham số truyền vào hàm `factorial`.
  - `$v0`: Giá trị giai thừa trả về từ hàm `factorial`.

2. Đoạn mã Assembly MIPS then chốt:

```
# 1. Bắt ngoại lệ số âm và tràn số 32-bit
    bltz $s0, error_neg         # Nếu N < 0 -> Báo lỗi theo yêu cầu đề bài
    li   $t9, 12
    bgt  $s0, $t9, error_ovf    # Nếu N > 12 -> Báo lỗi tràn số 32-bit

# 2. Thủ tục đệ quy factorial
factorial:
    ble  $a0, 1, fact_base      # Base Case: N <= 1 -> trả về 1

    # Tạo Stack Frame (cấp phát 8 bytes)
    addi $sp, $sp, -8
    sw   $ra, 4($sp)            # Lưu $ra
    sw   $a0, 0($sp)            # Lưu n

    addi $a0, $a0, -1           # n = n - 1
    jal  factorial              # Gọi đệ quy tính (n - 1)!

    # Phục hồi Stack Frame
    lw   $a0, 0($sp)            # Rút lại n
    lw   $ra, 4($sp)            # Rút lại $ra
    addi $sp, $sp, 8

    mul  $v0, $a0, $v0          # $v0 = n * (n - 1)!
    jr   $ra

fact_base:
    li   $v0, 1                 # 0! = 1, 1! = 1
    jr   $ra
```

3. Kết quả kiểm thử trên MARS 4.5:

STT	Mục đích kiểm thử	Dữ liệu vào N	Kết quả thực tế trên MARS	Đánh giá
TC01	Đề bài: số dương ($N = 5$)	5	`120`	**PASS**
TC02	Đề bài: số âm ngoại lệ	-2	`A factorial for your number can not be found.`	**PASS**
TC03	Biên cơ sở $0!$	0	`1`	**PASS**
TC04	Biên cơ sở $1!$	1	`1`	**PASS**
TC05	Biên cực đại 32-bit ($12!$)	12	`479001600`	**PASS**
TC06	Ngoại lệ tràn số 32-bit	13	`Error: Input exceeds 32-bit signed integer limit (Max N = 12; 12! = 479,001,600)!`	**PASS**

Bảng tổng kết cơ chế xử lý ngoại lệ trong toàn bộ Lab 2:

Bài tập	Loại ngoại lệ có thể gặp	Hậu quả nếu không bắt	Giải pháp trong mã MIPS	Tương đương trong C# / C / Java
Cả 4 bài	Nhập chuỗi ký tự, chữ cái hoặc số vượt 32-bit	Dữ liệu rác, phá hủy logic tính toán	Tầng 1: `syscall 5` Hardware Trap (`invalid integer input`)	`FormatException` / `InputMismatchException`
Bài 1 & 3	Căn lề ô nhớ không chia hết cho 4	Crash phần cứng `Alignment Exception` khi truy xuất RAM	Chỉ thị `.align 2` trước mảng `array`	Data Structure Memory Alignment
Bài 1	Phần tử là số âm ($x < 0$)	Lệnh `rem` cho số dư âm, làm tròn sai chiều trục số	Chuẩn hóa số dư: nếu $r < 0$ thì $r = r + 4$ về $[0, 3]$	`((x % 4) + 4) % 4` (Modulo chuẩn)
Bài 1	Phần tử cực đại $2^{31} - 1$ làm tròn lên	Hardware Trap: `Arithmetic overflow` khi `addi`	Kiểm tra $x > 2147483644$, báo lỗi `error_overflow`	`OverflowException`
Bài 2	Số nhập $a \leq 0$ hoặc $b \leq 0$	Sai định nghĩa GCD/LCM, gây chia cho 0	Chặn bằng `blez $s0` và `blez $s1`, báo lỗi thoát an toàn	`ArgumentOutOfRangeException`
Bài 2	Tràn số khi nhân tính LCM	Silent Overflow, kết quả lật dấu âm sai lệch	`mult` kết hợp kiểm tra `HI != 0` và `LO < 0`	`checked { (a / gcd) * b }`
Bài 3	Cả 10 phần tử bằng nhau	Không tồn tại giá trị lớn thứ nhì	Quản lý cờ `has_max2 == 0` sau Lượt 2, báo lỗi an toàn	`InvalidOperationException`
Bài 3	Mảng toàn số âm	Gán mốc `max2 = 0` sẽ sai lệch (vì $0 >$ mọi số âm)	Chỉ gán mốc `max2 = arr[i]` khi gặp số đầu tiên $< max1$	Baseline initialization by first match
Bài 4	Người dùng nhập số âm $N < 0$	Giai thừa số âm không xác định, đệ quy vô tận	Chặn bằng `bltz $s0`, in đúng thông báo yêu cầu của đề	`ArgumentOutOfRangeException`
Bài 4	Tràn số 32-bit khi $N > 12$ ($13! > 2^{31} - 1$)	Tràn số nguyên 32-bit có dấu, kết quả bị âm	Chặn trước bằng `bgt $s0, 12, error_ovf`	`OverflowException`

Kết luận:

1. **Lệnh rẽ nhánh và nhảy:** Sử dụng thành thạo các lệnh rẽ nhánh có điều kiện (`bge` , `ble` , `beq` , `bne` , `blez` , `bltz`) và lệnh nhảy không điều kiện (`j`) để xây dựng các vòng lặp và câu lệnh rẽ nhánh `if-else` .
2. **Quy ước gọi thủ tục (Procedure Calling Convention):** Nắm rõ vai trò của các thanh ghi truyền tham số `$a0-$a3` , thanh ghi trả về kết quả `$v0-$v1` , thanh ghi lưu địa chỉ trở về `$ra` , và ý thức bảo toàn các thanh ghi lưu trữ `$s0-$s7` .
3. **Cơ chế Ngăn xếp và Đệ quy:** Nắm vững nguyên tắc hoạt động của ngăn xếp (Stack) và con trỏ `$sp` , cách thiết lập và giải phóng Stack Frame để cất giữ `$ra` cùng các tham số cục bộ trong thủ tục đệ quy (Bài 4), đồng thời phân biệt được ưu thế hiệu năng của thủ tục lá không cần dùng Stack (Bài 2).
4. **Kiểm soát ngoại lệ và trường hợp biên:** Xử lý đầy đủ các lỗi biên, dữ liệu âm, mảng đồng nhất và tràn số nguyên 32-bit, đảm bảo chương trình chạy ổn định và an toàn trong mọi trường hợp kiểm thử.